

Control Flow Integrity

Project Summary

Institution: University of Texas at Dallas – Team b00n

Team Members:

Ian Brown	ixb130430@utdallas.edu
Anastasia Hudman	sxh122630@utdallas.edu
Joel Seida	jps140430@utdallas.edu
Jeremiah Shipman	jxs116830@utdallas.edu
Corrin Thompson	cnt130030@utdallas.edu

Document: Final Report

Keywords: Control Flow Integrity (CFI), Buffer Overflow, Memory Corruption Exploitation, Shadow Stack, Landing Point Instruction, Return Oriented Programming (ROP)

Project Description: NSA has developed techniques—landing point instructions and shadow stack—to enforce control flow integrity. This project will demonstrate the strength of these mitigations through attempted exploitation.

Deliverables:

- Proposal & Literature Review
- Progress Report & Presentation
- Final Report & Presentation
- Documentation & Code (<https://github.com/UTDInsure>)

Control Flow Integrity

Executive Summary

Institution:	University of Texas at Dallas – Team b00n	
Team Members:	Ian Brown	ixb130430@utdallas.edu
	Anastasia Hudman	sxh122630@utdallas.edu
	Joel Seida	jps140430@utdallas.edu
	Jeremiah Shipman	jxs116830@utdallas.edu
	Corrin Thompson	cnt130030@utdallas.edu

Keywords: Control Flow Integrity (CFI), Buffer Overflow, Memory Corruption Exploitation, Shadow Stack, Landing Point Instruction, Return Oriented Programming (ROP)

Summary:

Control flow integrity refers to the protection of the sequence of instructions in a program. Currently, existing control flow integrity mitigations only make it harder for the attacker to gain a foothold, rather than preventing access entirely. In order to address this issue, the NSA has created techniques that can be overlaid in a binary to enforce control flow integrity. These techniques include the landing point instructions and the shadow stack. The goal of this project involves the demonstration of the strength or weakness of these techniques through attempted exploitation of a suspected vulnerability. Analysis tools will be used to determine if the mitigations can be bypassed using the vulnerability. If successful, a proof of concept will be used to demonstrate the complete attack.

Table of Contents

Introduction	4
Problem Statement.....	4
Purpose Statement	4
Motivation.....	4
Impact	4
Literature Review	5
Methods and Procedures.....	7
Method Characterization	7
Procedures & Plan Overview	7
Schedule.....	7
Deliverables.....	8
Results to Date	9
Detailed Findings.....	9
Task 1: Develop a tool that will split gadget files into individually testable components.....	9
Task 2: Manually assessing feasibility of gadgets	10
Task 3: Chain gadgets.....	10
Issues.....	10
Conclusions and Recommendations	11
Future Work	11
References	12
Team	12
Biographical sketches.....	12
Ian Brown	12
Anastasia Hudman	13
Joel Seida.....	13
Jeremiah Shipman.....	13
Corrin Thompson	14
Tasking	14

Introduction

Problem Statement

Malicious software can take advantage of vulnerabilities in a program by redirecting its flow of execution (the control flow). Control flow integrity (CFI) is designed to prevent these types of attacks. NSA developed two CFI techniques: landing point instructions and shadow stack. The objective is to test one of the landing point instruction techniques in order to demonstrate its strengths and weaknesses.

Purpose Statement

This project aims to test the resilience of the landing point instruction safety technique by chaining gadgets (small snippets of machine code) in order to manipulate a return address in a meaningful way. If a return address can be manipulated, then the CFI technique can be shown to be ineffective.

Motivation

Control flow integrity is vital to the prevention of the spread of malware. There are many different implementations that attempt to prevent an attacker from hijacking program flow and taking it down an unintended path. However, in the current situation, most control flow integrity mechanisms do not address the issue of control flow directly. Instead, these mitigations only serve to make it harder for an attacker to maintain control, but do not entirely prevent the attacker from gaining control in the first place. Other solutions must be considered in order to properly maintain control flow integrity.

The two techniques developed by NSA (landing point instructions and shadow stack) are a crucial step in preventing the next level of malware attacks. However, they cannot be implemented without thorough testing and evaluation. This project will attempt to uncover potential flaws in the design of these mechanisms in order to help demonstrate their weakness, or perhaps their strength. If these techniques are proven worthy, they can become a boon to our society.

Impact

The research conducted in this semester will provide further evidence to prove or disprove the security that the shadow stack and landing point instructions provide. While the scope of this project has been limited to the study of gadget chaining, the tool might be able to be expanded for further use by other teams.

Demonstrating the effectiveness of these CFI methodologies could facilitate the development of new CFI methodologies and make them commonplace in future software products. Eventually, if these CFI techniques lead to a system that is completely secure against ROP/COP/JOP attacks, this research will have been a boon to society.

Literature Review

Relevance

Control flow Integrity (CFI) is the series of security techniques to prevent arbitrary code execution by altering the flow of the program. Attackers may be able to alter the control flow of a program using variations of a buffer overflow attack, where a return address is overwritten, and points to malicious code to be executed ^[1].

Despite buffer overflows being fairly basic and well-known vulnerabilities, they are prevalent in modern software. Between 1988 and 2012, buffer overflows accounted for the largest portion of reported vulnerabilities (about 14%). Of vulnerabilities with a "high" severity rating, buffer overflows make up 23%, and of critical vulnerabilities they consist of 35% ^[2]. A well-known example of this attack is the Twilight Hack used to homebrew a Wii console (developed in 2008). This attack overloads the value of a character's name, which when loaded into memory, spills into adjacent buffers, allowing arbitrary programs to be executed ^[4].

Developing CFI attacks that are infeasible to break is crucial to protecting company and organizational software as new attack methods are developed.

Control Flow Integrity Methods

In June 2016, Intel released a document detailing a new Control-flow Enforcement Technology (CET) to protect against return-oriented programming (ROP) attacks and call/jmp-oriented programming (COP/JOP) ^[3]. These types of attack techniques are typically able to divert control flow from its original target address to a different target by changing the data stack or register. The techniques also have a module of code that possesses execution privileges and code containing control transfer instructions that rely on data either in the return stack or in a target address register.

Intel's CET contains two mechanisms, shadow stack and indirect branch tracking, to protect against ROP and COP/JOP style attacks. The Shadow Stack technique is a return address protection used to protect against ROP attacks. The indirect branch tracking technique provides free branch defense to protect against JOP attacks.

NSA Proposed Methods

NSA proposes a protected shadow stack in combination with using landing point instructions. The protected shadow stack is in a secure area of memory, separate from the regular stack. When a return address is stored in the regular stack, it is also stored in the shadow stack. Upon returning, the return value is compared to the value in the shadow stack.

Landing point instructions are newly proposed instructions for the x86 architecture. Whenever a branch (jump, call, return) statement occurs, the branch's destination must be a corresponding landing point instruction. With these two features, NSA hopes to eliminate ROP, JOP, and COP attack techniques^[5].

Previous Group Work

The group from CMU that worked on the project in Fall 2016 focused on a method of ranking gadgets by intensity of side-effects. Over the course of the semester, they produced a tool capable of accomplishing this goal. Due to the short time frame of the project, they were unable to add all intended features of the tool.

The group from DSU that worked on CFI in Spring 2016 focused on attempting to prove Turing-completeness for gadgets in supplied binaries. While their findings were inconclusive, they did show that Turing-complete gadget sets would be hard to find.

Team Approach and Justification

The team will create a tool capable of chaining together gadgets. The tool will attempt to circumvent NSA's control flow integrity techniques (landing point instructions and the shadow stack) in order to gain control of the application using this process.

To create this tool, the team will first analyze and attempt to manually determine potentially feasible gadget chains. The tool will then be created in order to automatically execute these pre-written chains. If time permits, the tool may be expanded to accommodate the usage of more gadgets. Otherwise the source code will be provided for future teams to expand on this work.

Methods and Procedures

Method Characterization

The main methods used were exploratory and constructive. An exploratory approach was used to identify and define the original problem of attempting to chain gadgets. Constructive research was incorporated to test potential gadgets that could be chained together to produce unexpected changes to the flow of control by attempting to manipulate registers.

Procedures & Plan Overview

The goal of this project was to prove the feasibility (or infeasibility) of breaking the landing point instruction CFI method designed by the NSA. To do this, the team focused on gaining control of a single register (the EAX register). Gaining control of the register requires chaining together gadgets in a specific format. Accomplishing this goal required sifting through a wide array of pre-defined gadgets.

The team developed a series of tools that scanned these gadgets, and selected the gadgets that incremented and decremented the EAX register by one. The team then sought out to find a gadget that would be able to loop these gadgets, chaining them together and gaining control of EAX.

Due to time constraints, the team could not develop a tool to find such a gadget. Contents of this gadget were theorized and listed, so that future groups may continue this work and discover these gadgets (if they exist).

Schedule

Below is the schedule listing weekly goals for the semester.

Feb 17: Submit Proposal

Feb 24: Review project documentation, study how different gadgets work

Mar 3: Draft tool design, define set of gadgets to be used

Mar 10: Finalize tool design and assign roles to members, implementation of design

Mar 17: Implementation of design

Mar 24: Prepare presentation and progress report

Mar 31: Submit progress report and presentation

Apr 7: Analyze individual gadgets

Apr 14: Test gadgets against provided Linux binary file

Apr 21: Additional testing and final report draft

May 5: Submit final report and presentation

Deliverables

- Proposal & Literature Review
- Progress Report & Presentation
- Final Report & Presentation
- Documentation & Code

Limitations and Delimitations

The main limitation of this project involved analyzing thousands of gadgets within a limited time constraint. Other limitations stemmed from the difficulty in breaking theoretical implementations of defenses. These limitations restricted the scope of the overall project, as mentioned in the issues section.

The main delimitation for the project involved assuming the shadow stack technique is unbreakable and focusing on instead manipulating the outcome of the landing point instruction technique. More specific delimitations include the particular gadgets that were analyzed. Because of the high number of varying gadgets, a tool had to be developed to split gadget files into individually testable components. The tool sorted gadgets based on ease of testing, filtering out gadgets with jump operations.

Results to Date

Findings Overview

The team decided to research exploits which used Jump Oriented Programming (JOP) against a system using Landing Point Instructions. The team assumed Return Oriented Programming (ROP) was no longer viable due to the Shadow Stack. Searching through a total of roughly 630,000 gadgets, 72 gadgets were discovered that increment or decrement EAX by one or call via the RAX register. This was part of the plan to create a string of gadgets that would loop an EAX increment to a desired value. While searching, the degree to which landing point instructions restrict what gadgets can be used was revealed. With no landing point instructions, an attacker could form gadgets from whatever assembly code snippets he or she desired. Now the attacker would have a widely reduced selection to choose from. It appears far more difficult to construct gadgets using Jump Oriented Programming (JOP). The team was unable to find a "looping" gadget that would work with one of the previously found gadgets. First of all, the structure of this gadget is more advanced than the previous ones. Also, due to the landing point instructions, a gadget had to be used entirely, or not at all. Often, undesired assembly instructions were attached like leeches to an otherwise useful gadget. These instructions corrupted the values of the registers that were needed. Without the defense of landing point instructions, these unwanted instructions could have been stripped out.

Detailed Findings

Task 1: Develop a tool that will split gadget files into individually testable components

Process:

A tool was written as a series of bash scripts to split and trim the gadgets. All files are hosted on GitHub. The tool was run over the course of a weekend. The stripped gadgets were divided into two categories: easy and hard. After that, each gadget was executed in a C program that tests whether or not the EAX register was incremented or decremented by one.

Findings:

Completion of this subtask resulted in a set of "easy" gadgets that contained no jump operations. These would be easier to assess by the team and determine register modifications. This also resulted in a set of gadgets that incremented or decremented the EAX register by one.

Task 2: Manually assessing feasibility of gadgets

Process:

Each member of the team analyzed gadgets and assessed their potential success in breaking the landing point instruction technique.

Findings:

Analysis of the gadgets showed that they did not contain many statements capable of modifying registers without returning to a predefined landing point. While the team could not prove the infeasibility of using the gadgets to break the landing point instruction methodology, it was obvious that doing so would be difficult.

Task 3: Chain gadgets

Process:

The goal of this task was to find a gadget that would successfully loop gadgets and execute them in sequence against the provided Linux binary. A tool would then be developed to automatically chain these gadgets together and test if whether the team gained control of the register.

Findings:

Unfortunately a looping gadget could not be discovered in time to create the testing tool. The implementation of the above step is left to future groups. Ultimately the team was able to theorize the components of this gadget and what it would look like. Details are discussed further in the conclusion section. In short, partial control of other registers are needed to gain control of another register, or a gadget must make insecure calls to the stack in order to gain control of a register.

Issues

Issues encountered:

- Difficulty in breaking theoretical implementations of defenses
- Several hundred thousand gadgets to analyze and test within limited time constraints

- Landing Point Instruction technique has already been thoroughly tested, making generation of original tests challenging

Issues regarding differences in team skillsets did arise as predicted. Transfer of knowledge has not been as efficient as desired, due to conflicting exam schedules and multiple external obligations by individual team members. In the following month, individual team member schedules become less hectic, and progress lost due to differing skillsets can be remedied within the allotted time.

Conclusions and Recommendations

The scope narrowed from effectively breaking the landing point instruction method to discovering a method of gaining control of a register. It can be assumed that if an attacker has control of a register, the control flow of the program can be effectively altered. From the analysis of the gadgets provided by the NSA, it has been concluded that there is a feasible way to gain control of the EAX register. However, it operates under the assumption that the attacker already has partial control of the stack. Attacks attempting to gain control of a register with no prior control would require the gadget to make insecure calls to the stack – which is unlikely to be implemented in widely distributed programs. Ultimately this means that it is possible to break the landing point instruction mechanism, but only under a specific set of circumstances.

Future Work

To complete the original goal of this project, future groups are recommended to find the looping gadget described in this document: a loop that makes a call to the stack, and also has a loop invariant mostly dependent on the stack. Lesser forms of this gadget could potentially prove successful. Finding some form of this gadget could lead towards a successful attack on the landing point instruction methodology and would provide a launch pad for developing attacks. Future teams could also formulate different theories on gadget-chains that could work. Gadgets that work with values on the stack seem like they could be useful as well.

References

- [1] M. Abadi, M. Budiu, Ú. Erlingsson and J. Ligatti, "Control-flow integrity principles, implementations, and applications", ACM Transactions on Information and System Security, vol. 13, no. 1, pp. 1-40, 2009.
- [2] Y. Younan, "25 years of vulnerabilities: 1988 - 2012," SourceFire, 2013.
- [3] Intel, "Control-flow Enforcement Technology Preview", intel.com, 2016. [Online]. Available: <https://software.intel.com/sites/default/files/managed/4d/2a/control-flow-enforcement-technology-preview.pdf>. [Accessed: Feb. 9, 2017].
- [4] "Twilight hack," in wiibrew.org, 2014. [Online]. Available: http://www.wiibrew.org/wiki/Twilight_Hack. [Accessed: Feb. 9, 2017].
- [5] NSA, "Hardware Control Flow Integrity for an IT Ecosystem," in github.com, GitHub, 2015. [Online]. Available: <https://github.com/iadgov/Control-Flow-Integrity/blob/master/paper/Hardware%20Control%20Flow%20Integrity%20for%20an%20IT%20Ecosystem.pdf>. Accessed: [Accessed: Feb. 9, 2017].

Team

Biographical sketches

Ian Brown

Ian Brown holds a Bachelor's degree in Software Engineering with a specialization in computer networks and is currently pursuing a Master's degree in Computer Science with a specialization in information assurance. He is a recipient of the NSF's Scholarship for Service. Ian has a wide variety of security experience including binary exploitation, web/application/network penetration testing, and secure programming. He has completed coursework in computer networks, operating systems, MIPS assembly language, operating systems, information security, and network security. Beyond the classroom, Ian has also held the secretary and president positions within the Computer Security Group at UT Dallas and is studying for the OSCP certification.

Anastasia Hudman

Anastasia Hudman holds a Bachelor's degree in Software Engineering and is currently pursuing a Master's degree in Computer Science with a specialization in information assurance. She is a CyberCorps: Scholarship for Service recipient and has completed an internship for the Department of Defense as a Blue Team Forensic Analyst. She has experience in memory analysis, virtual machines, software development, and project management. She has taken courses in data and applications security, information security, operating systems, computer networks, and database design. Her knowledge of programming languages includes Java, C++, and C. She held officer roles in the Computer Security Group at UT Dallas and helped create challenges for a Capture the Flag competition. She also volunteered at the Women in Cybersecurity Conference in Spring 2016.

Joel Seida

Joel Seida holds a Bachelor's degree in Computer Science and is currently pursuing a Master's degree in Computer Science with a specialization in Information Assurance. He is a recipient of the NSF's Scholarship for Service and has completed an internship for Sandia National Labs as a part of the Center for Cyber Defenders. He has also taken coursework in data and applications security, information security, operating systems, computer networks, distributed computing, network security, database design, algorithm design, and is currently taking a class on computation theory.

Jeremiah Shipman

Jeremiah Shipman holds a Bachelor's degree in Computer Science and is currently pursuing a Master's degree in Computer Science with a specialization in Information Assurance. He is a recipient of the NSF's Scholarship for Service and has completed an internship for the Los Alamos National Laboratory. He has experience solving cyber-security problems in CTF (capture-the-flag) competitions (including ones involving buffer overflows). He has related course experience in operating systems, MIPS assembly language, information security, and network security. He is also currently preparing for the OSCP online course and certification.

Corrin Thompson

Corrin Thompson is a senior-level software engineering major, with a specialization in information assurance. She is currently a fast track student and working on a master's degree in computer science while finishing undergraduate studies. Since freshman year, Corrin has been involved with a local student organization on campus: the Computer Security Group (CSG). Her work in the group has given her experience in topical areas of computer security, as well as leadership and communication. Her current specializations are digital forensics and web security. In addition she has also worked on the Security & Architecture team with Liberty Mutual Insurance, performing vulnerability analysis on pre-production code and developing tools to aid in incident response.

Tasking

Member roles will vary based on the phases of the project as the group learns each members' strengths and weaknesses. The three software engineering majors, Ian Brown, Anastasia Hudman, and Corrin Thompson, will handle most of the documentation, requirements analysis, and testing as the project progresses. The two computer science majors, Joel Seida and Jeremiah Shipman, will lead in design and implementation of code. Every member will contribute in devising an attack to attempt to bypass the mitigations.